DC - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continous color palettes
```

```
# load needed functions
# source(paste0(RootDir,"/src/function_library/sp_functions.R"))
# do this in the calling R script
```

```
# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat," - ",CurTask," - ",RunLabel," - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE
```

```
CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres
```

```
# done in calling script
# print(paste0("Using root dir: ",RootDir))
# RunDir <- paste0(paste0(RootDir,"/output/",RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir,showWarnings = TRUE)
#   print(paste0("created run directory: ",RunDir))
# }
```

```
# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir,showWarnings = TRUE)
#   print(paste0("created participant directory: ",PatientDir))
# }
# TablesDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir,showWarnings = TRUE)
#   print(paste0("created tables directory: ",TablesDir))
# }
# FigDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir,showWarnings = TRUE)
#   print(paste0("created figures directory: ",FigDir))
# }
# ReportsDir <- paste0(RootDir,"/output/",RunLabel,"/",CurPat,"/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir,showWarnings = TRUE)
#   print(paste0("created reports directory: ",ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())
```

```
ModelDatFilename<-paste0(RootDir,"/data/",CurPat,"/",CurPat,"_",CurTask,"_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ",ModelDatFilename))
  print(sprintf("\troot dir: ",RootDir))
}
PosDat<-read.csv(ModelDatFilename)
```

```
# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values",OrigDataLen))
```

```
## [1] "Original data has 4057 values"
```

```
if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values",DataLen))
  print(sprintf("%d values eliminated.",OrigDataLen - DataLen))
}
```

```
## [1] "Data without fractional preserved values has 3572 values"
## [1] "485 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite

# get counts of syllable components in different serial positions
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syl

syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))

kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	481	27	109	NA	NA	617
2	50	NA	342	82	90	564
3	270	NA	130	170	13	583
4	248	NA	187	54	30	519
5	197	NA	157	64	28	446
6	183	1	103	53	20	360
7	141	NA	85	25	16	267
8	78	NA	36	23	4	141
9	68	NA	2	NA	5	75

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7795786	0.0437601	0.1766613	NA	NA	617
2	0.0886525	NA	0.6063830	0.1453901	0.1595745	564
3	0.4631218	NA	0.2229846	0.2915952	0.0222985	583
4	0.4778420	NA	0.3603083	0.1040462	0.0578035	519
5	0.4417040	NA	0.3520179	0.1434978	0.0627803	446
6	0.5083333	0.0027778	0.2861111	0.1472222	0.0555556	360
7	0.5280899	NA	0.3183521	0.0936330	0.0599251	267
8	0.5531915	NA	0.2553191	0.1631206	0.0283688	141
9	0.9066667	NA	0.0266667	NA	0.0666667	75

```
# get percentages only from summary table
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
syll_comp_perc <- syll_comp_perc %>% rename("Coda" ="1","Satellite"="S")

# pivot to long format for plotting
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
            names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda","Sa
```

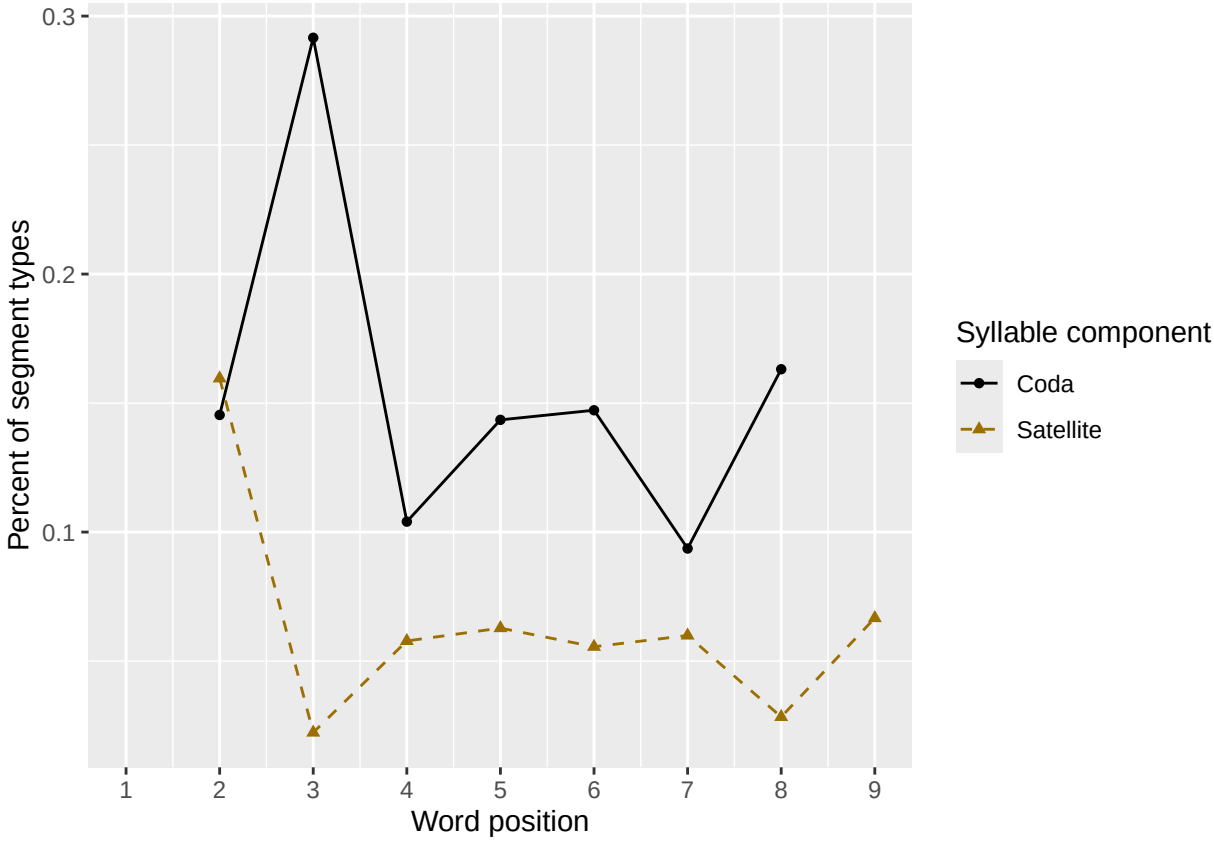
```
# plot satellite and coda occurances across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                         aes(x=pos,y=percent,group=syll_component,
                             color=syll_component,
                             linetype = syll_component,
                             shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype_
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_pos_of_syll_complexities_in_stimuli.png"),plot=sy
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_point()`)
```

```
syll_comp_plot
```

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
## Removed 3 rows containing missing values or values outside the scale range (`geom_point()`).
```




```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```
min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,"_preserved_percentages_by_len_
pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.596 0.824 0.604 NA     NA     NA     NA     NA     NA
## 2       5 0.614 0.761 0.614  0.529 NA     NA     NA     NA     NA
## 3       6 0.557 0.583 0.472  0.611  0.312 NA     NA     NA     NA
## 4       7 0.481 0.46  0.385  0.495  0.556  0.416 NA     NA     NA
## 5       8 0.545 0.512 0.402  0.370  0.482  0.552  0.367 NA     NA
## 6       9 0.494 0.384 0.218  0.261  0.36   0.394  0.5    0.351 NA
## 7      10 0.456 0.409 0.24   0.303  0.268  0.292  0.315  0.453  0.427
```

```
# len/pos table
pos_len_N <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved),N=n()) %>% dply
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                 "_N_by_len_position.csv"))
pos_len_N_table
```

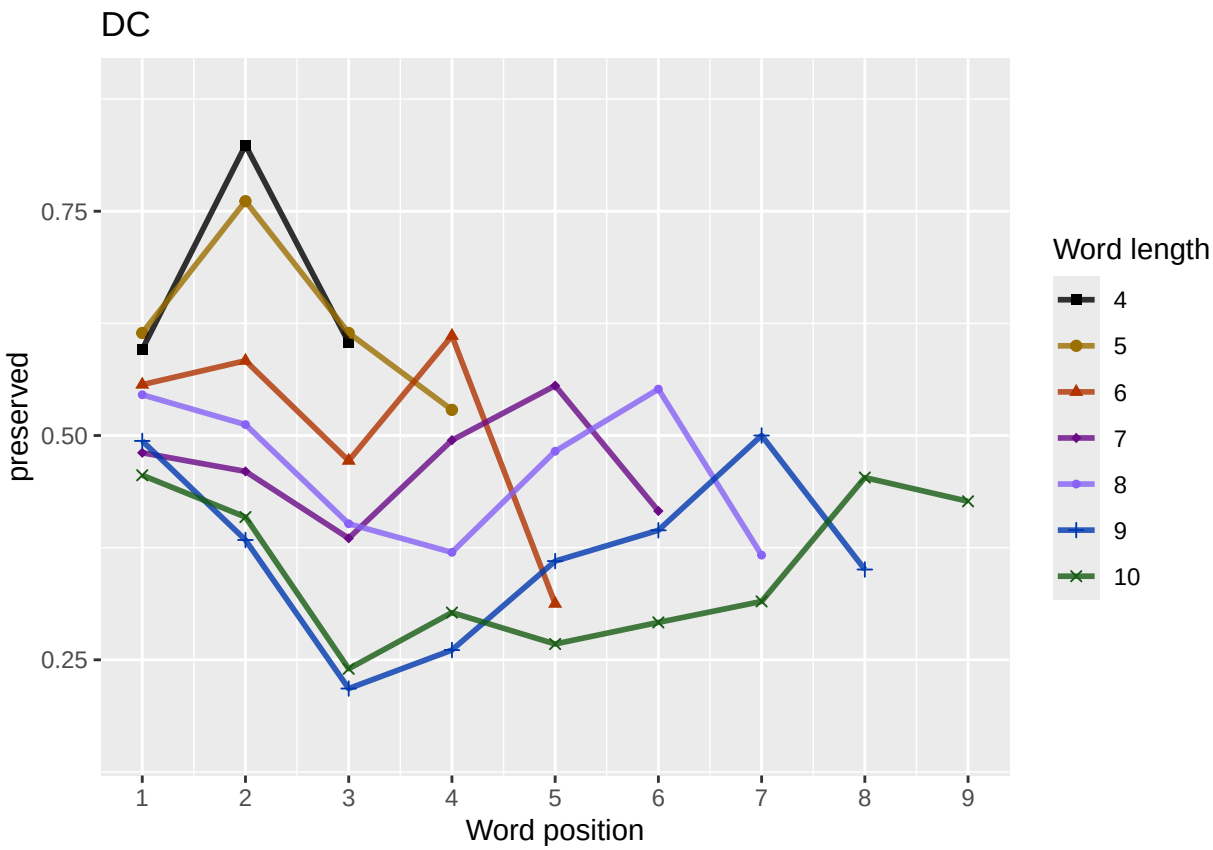
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##     <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1       4    52    51    53    NA    NA    NA    NA    NA    NA
## 2       5    70    67    70    70    NA    NA    NA    NA    NA
## 3       6    97    84    89    90    96    NA    NA    NA    NA
## 4       7   104   100    96    95    90   101    NA    NA    NA
## 5       8   132   123   122   119   114   116   120    NA    NA
## 6       9    83    73    78    69    75    71    74    77    NA
## 7      10    79    66    75    76    71    72    73    64    75
```

```
obs_linetypes <- c("solid","solid","solid","solid",
                   "solid","solid","solid","solid","solid")
pred_linetypes <- "dashed"
pos_len_summary$stimlen<-factor(pos_len_summary$stimlen)
pos_len_summary$pos<-factor(pos_len_summary$pos)
# len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimle
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                           paste0(CurPat),
                                           NULL,
                                           c(min_preserved,max_preserved),
                                           palette_values,
                                           shape_values,
                                           obs_linetypes,
                                           pred_linetypes)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot,device="png",unit="cm",width=15,height=11)
len_pos_plot
```



Length and position

```
# length and position

LPModelEquations<-c("preserved ~ 1",
            "preserved ~ stimlen",
            "preserved ~ pos",
            "preserved ~ stimlen + pos",
            "preserved ~ stimlen*pos",
            "preserved ~ I(pos^2)+pos",
            "preserved ~ stimlen + I(pos^2) + pos",
            "preserved ~ stimlen * (I(pos^2) + pos)"
)

LPRes<-TestModels(LPModelEquations,PosDat)
```

```
## ****************************
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
```

```
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     1.95996     -0.21163      0.03191     -0.29622
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3568 Residual
## Null Deviance:       4922
## Residual Deviance: 4794  AIC: 4802
## log likelihood:  -2396.917
## Nagelkerke R2:  0.04714216
## % pres/err predicted correctly:  -1707.346
## % of predictable range [ (model-null)/(1-null) ]:  0.03544537
## ***************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##          1.09775          -0.10956          -0.05299           0.31926           0.00956          -0
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3566 Residual
## Null Deviance:       4922
## Residual Deviance: 4790  AIC: 4802
## log likelihood:  -2395.156
## Nagelkerke R2:  0.04841378
## % pres/err predicted correctly:  -1705.692
## % of predictable range [ (model-null)/(1-null) ]:  0.03637891
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##     2.14330     -0.28331     -0.26951      0.02864
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3568 Residual
## Null Deviance:       4922
## Residual Deviance: 4807  AIC: 4815
## log likelihood:  -2403.439
## Nagelkerke R2:  0.04242356
## % pres/err predicted correctly:  -1713.4
## % of predictable range [ (model-null)/(1-null) ]:  0.03202704
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
```

```
##     1.40438     -0.19295     -0.02753
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3569 Residual
## Null Deviance:       4922
## Residual Deviance: 4814  AIC: 4820
## log likelihood:  -2406.891
## Nagelkerke R2:  0.03991846
## % pres/err predicted correctly:  -1716.801
## % of predictable range [ (model-null)/(1-null) ]:  0.03010656
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      1.4019      -0.2062
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3570 Residual
## Null Deviance:       4922
## Residual Deviance: 4816  AIC: 4820
## log likelihood:  -2408.174
## Nagelkerke R2:  0.03898668
## % pres/err predicted correctly:  -1718.102
## % of predictable range [ (model-null)/(1-null) ]:  0.02937228
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     0.40138      0.01951     -0.25186
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3569 Residual
## Null Deviance:       4922
## Residual Deviance: 4885  AIC: 4891
## log likelihood:  -2442.541
## Nagelkerke R2:  0.01376633
## % pres/err predicted correctly:  -1751.773
## % of predictable range [ (model-null)/(1-null) ]:  0.01036081
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##     0.13748     -0.08444
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3570 Residual
```

```
## Null Deviance:       4922
## Residual Deviance: 4893  AIC: 4897
## log likelihood:  -2446.483
## Nagelkerke R2:  0.01084209
## % pres/err predicted correctly:  -1755.699
## % of predictable range [ (model-null)/(1-null) ]:  0.008144236
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##      -0.183
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3571 Residual
## Null Deviance:       4922
## Residual Deviance: 4922  AIC: 4924
## log likelihood:  -2461.025
## Nagelkerke R2:  2.968884e-16
## % pres/err predicted correctly:  -1770.124
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                         AIC=LPRes$AIC,
                         row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)
kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen + I(pos^2) + pos	4801.834	4	1.000000	1.000000	0.0558808	4.047142	-2.599583	-0.2116299	2.962176	NA	0.0319065	NA
preserved ~ stimlen * (I(pos^2) + pos)	4802.311	0.4769499	0.7878284	0.4402451	0.0484138	4.097745	-6	-0.1095552	0.3192592	0.0709556	-0.0529853	0.0095598
preserved ~ stimlen * pos	4814.877	13.042696	0.0014707	0.0008224	0.0424236	4.361433	-3.001	-0.2833120	2.695120	0.0286418	NA	NA
preserved ~ stimlen + pos	4819.782	17.947754	0.0001267	7.08e-05	0.0399185	4.043845	-2.929	-0.1929508	2.675328	NA	NA	NA

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen	4820.34	18.51271	7.5e-05	5.5e-05	0.4038986	1.74018558	-0.2062240	NA	NA	NA	NA
preserved ~ I(pos^2) + pos	4891.08	89.24740	5.0e-21	0.0e+00	0.0137663	1.34013786	NA	-0.2518602	NA	0.0195077	NA
preserved ~ pos	4896.96	95.13212	2.6e-21	0.0e+00	0.0108421	1.11374835	NA	-0.0844419	NA	NA	NA
preserved ~ 1	4924.04	122.21507	7.1e-27	0.0e+00	0.0000000	0.00000000	-0.1830401	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen + I(pos^2) + pos"
```

```
print(BestLPModel)
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     1.95996     -0.21163      0.03191     -0.29622
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3568 Residual
## Null Deviance:       4922
## Residual Deviance: 4794  AIC: 4802
```

```
PosDat$LPFitted<-fitted(BestLPModel)
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],
                         NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

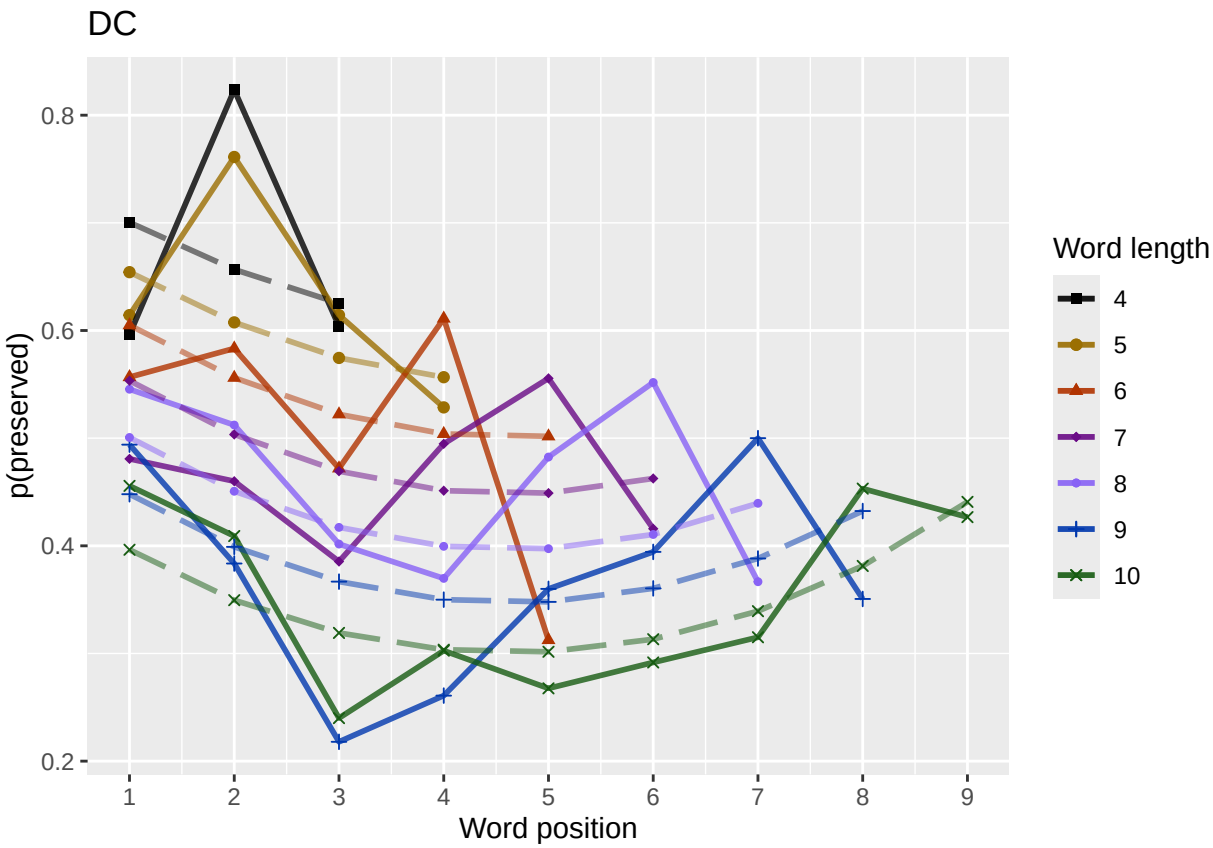
```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##     <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1       4 0.700 0.657 0.625 NA     NA     NA     NA     NA     NA
## 2       5 0.654 0.608 0.575  0.557 NA     NA     NA     NA     NA
## 3       6 0.605 0.556 0.522  0.504  0.502 NA     NA     NA     NA
## 4       7 0.553 0.503 0.469  0.451  0.449  0.463 NA     NA     NA
## 5       8 0.501 0.451 0.417  0.400  0.397  0.411  0.440 NA     NA
## 6       9 0.448 0.399 0.367  0.350  0.348  0.360  0.388  0.432 NA
## 7      10 0.396 0.350 0.319  0.303  0.302  0.313  0.339  0.381  0.441
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitte
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
                                          paste0(PosDat$patient[1]),
                                          "LPFitted",
                                          NULL,
                                          palette_values,
                                          shape_values,
                                          obs_linetypes,
                                          pred_linetypes = c("longdash")
                                          )
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_percent_preserved_by_length_pos_wfit.png"),plot=
fitted_len_pos_plot
```



length and position without fragments to see if this changes position^2 influence

```
# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models
```

```
# number responses
resp_num<-0
prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array
```

```
# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag
```

```
## # A tibble: 1 x 2
##   frag_sum     N
##      <int> <int>
## 1       14   648
```

```
num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100
```

```
## Warning: Unknown or uninitialised column: `percent_with_frag`.
```

```
print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
              num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))
```

```
## [1] "The number of responses with fragments was 14 / 648 = 2.16 percent"
```

```
write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_percent_fragments.csv"),row.names = FALSE)
```

```
# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)
```

```
## ****************************
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     1.97836     -0.21478      0.03275     -0.29594
##
## Degrees of Freedom: 3549 Total (i.e. Null);  3546 Residual
## Null Deviance:       4895
## Residual Deviance: 4768  AIC: 4776
## log likelihood:  -2384.079
## Nagelkerke R2:  0.04701591
## % pres/err predicted correctly:  -1698.739
## % of predictable range [ (model-null)/(1-null) ]:  0.03532633
```

```
## ***************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##          1.13790          -0.11226          -0.04025           0.26777           0.00842          -0
##
## Degrees of Freedom: 3549 Total (i.e. Null);  3544 Residual
## Null Deviance:       4895
## Residual Deviance: 4765  AIC: 4777
## log likelihood:  -2382.647
## Nagelkerke R2:  0.04805629
## % pres/err predicted correctly:  -1697.426
## % of predictable range [ (model-null)/(1-null) ]:  0.03607149
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##     2.09729     -0.27979     -0.24601      0.02669
##
## Degrees of Freedom: 3549 Total (i.e. Null);  3546 Residual
## Null Deviance:       4895
## Residual Deviance: 4783  AIC: 4791
## log likelihood:  -2391.544
## Nagelkerke R2:  0.0415814
## % pres/err predicted correctly:  -1705.714
## % of predictable range [ (model-null)/(1-null) ]:  0.03136814
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      1.4081      -0.2056
##
## Degrees of Freedom: 3549 Total (i.e. Null);  3548 Residual
## Null Deviance:       4895
## Residual Deviance: 4790  AIC: 4794
## log likelihood:  -2395.217
## Nagelkerke R2:  0.03889857
## % pres/err predicted correctly:  -1709.358
## % of predictable range [ (model-null)/(1-null) ]:  0.02929984
## ***************************
## model index:  4
##
```

```
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##     1.40986     -0.19576     -0.02036
##
## Degrees of Freedom: 3549 Total (i.e. Null);  3547 Residual
## Null Deviance:       4895
## Residual Deviance: 4789  AIC: 4795
## log likelihood:  -2394.522
## Nagelkerke R2:  0.03940655
## % pres/err predicted correctly:  -1708.648
## % of predictable range [ (model-null)/(1-null) ]:  0.02970261
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     0.39771      0.02019     -0.25161
##
## Degrees of Freedom: 3549 Total (i.e. Null);  3547 Residual
## Null Deviance:       4895
## Residual Deviance: 4862  AIC: 4868
## log likelihood:  -2430.994
## Nagelkerke R2:  0.01247634
## % pres/err predicted correctly:  -1744.439
## % of predictable range [ (model-null)/(1-null) ]:  0.009389945
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##     0.12509     -0.07848
##
## Degrees of Freedom: 3549 Total (i.e. Null);  3548 Residual
## Null Deviance:       4895
## Residual Deviance: 4870  AIC: 4874
## log likelihood:  -2435.191
## Nagelkerke R2:  0.009341865
## % pres/err predicted correctly:  -1748.616
## % of predictable range [ (model-null)/(1-null) ]:  0.007019032
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
```

```
## Coefficients:
## (Intercept)
##     -0.1717
##
## Degrees of Freedom: 3549 Total (i.e. Null);  3549 Residual
## Null Deviance:       4895
## Residual Deviance: 4895  AIC: 4897
## log likelihood:  -2447.64
## Nagelkerke R2:  0
## % pres/err predicted correctly:  -1760.984
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```r
NoFragBestLPModel<-NoFrag_LPRes$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPRes$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPRes$Model,
                       AIC=NoFrag_LPRes$AIC,
                       row.names = NoFrag_LPRes$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPRes$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPRes$CoefficientValues, by='row.names',sort=FALSE
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen + I(pos^2) + pos	4776.159	0.000000	1.000000	0.6378646	0.0470159	1.9783588	-0.2147837	-0.2959424	NA	0.0327506	NA
preserved ~ stimlen * (I(pos^2) + pos)	4777.291	1.134895	0.566970	0.3616507	0.0480563	1.1378986	-0.1122595	0.2677722	-0.0668210	-0.0402464	0.00842
preserved ~ stimlen * pos	4791.088	14.928623	0.0005732	0.0003656	0.0415824	1.0972943	-0.2797946	-0.2460087	0.0266907	NA	NA
preserved ~ stimlen	4794.434	18.275285	0.0001075	0.0000686	0.0388981	1.4081296	-0.2055611	NA	NA	NA	NA
preserved ~ stimlen + pos	4795.044	18.885401	0.0000793	0.0000506	0.0394065	1.4098583	-0.1957626	-0.0203615	NA	NA	NA
preserved ~ I(pos^2) + pos	4867.988	91.829182	0.0000000	0.0000000	0.0124763	0.3977104	NA	-0.2516063	NA	0.0201922	NA
preserved ~ pos	4874.382	98.222739	0.0000000	0.0000000	0.0093409	0.1250888	NA	-0.0784799	NA	NA	NA
preserved ~ 1	4897.280	121.121466	0.0000000	0.0000000	0.0000000	-0.1716881	NA	NA	NA	NA	NA

```
# plot no fragment data

NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
                          NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPFit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

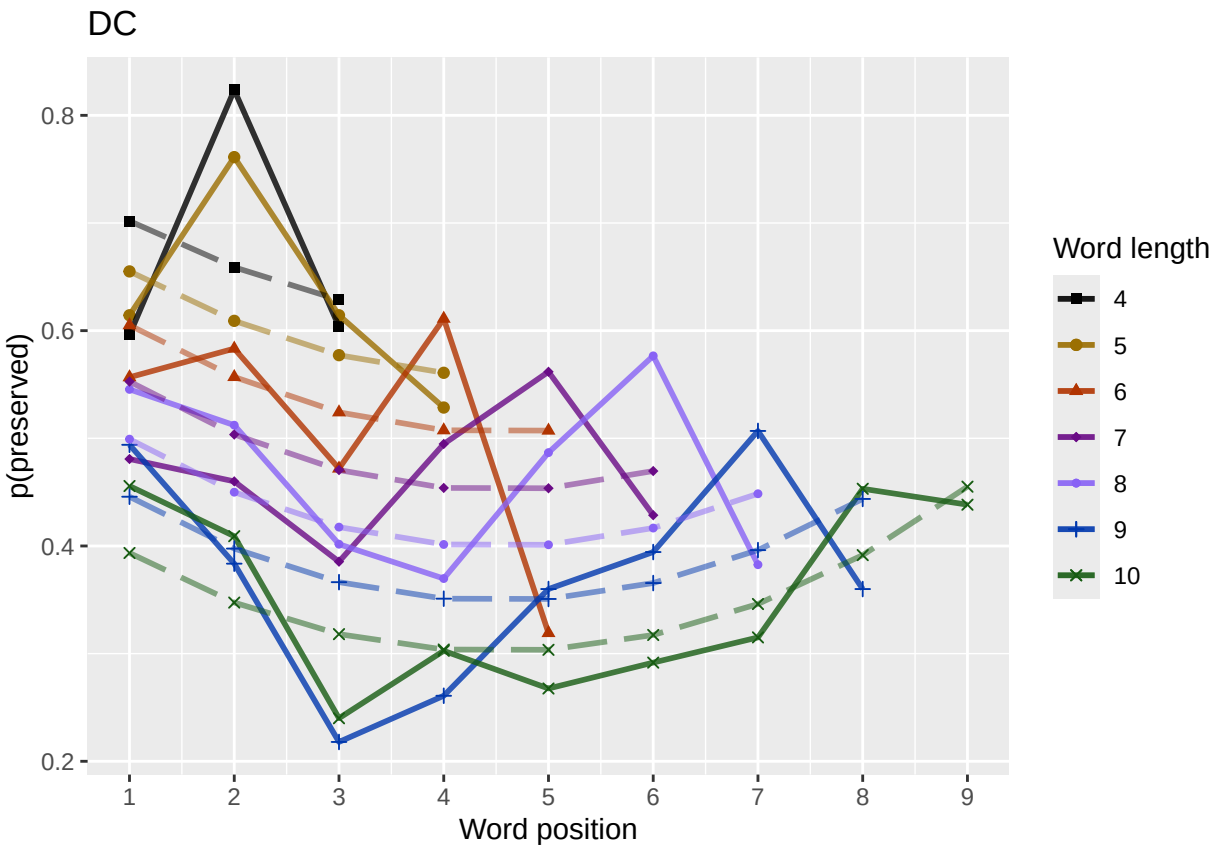
```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`    `4`    `5`    `6`    `7`    `8`    `9`
##     <int> <dbl> <dbl> <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
## 1       4 0.702 0.659 0.629 NA     NA     NA     NA     NA     NA
## 2       5 0.655 0.609 0.577  0.561 NA     NA     NA     NA     NA
## 3       6 0.605 0.557 0.524  0.507  0.507 NA     NA     NA     NA
## 4       7 0.553 0.503 0.470  0.454  0.454  0.470 NA     NA     NA
## 5       8 0.499 0.450 0.418  0.401  0.401  0.417  0.449 NA     NA
## 6       9 0.446 0.398 0.366  0.351  0.351  0.366  0.396  0.444 NA
## 7      10 0.393 0.347 0.318  0.304  0.304  0.317  0.346  0.392  0.455
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitte
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(pas

nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
                                          paste0(NoFragData$patient[1]),
                                          "LPFitted",
                                          NULL,
                                          palette_values,
                                          shape_values,
                                          obs_linetypes,
                                          pred_linetypes = c("longdash")
                                          )
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
              "no_fragments_percent_preserved_by_length_pos_wfit.png"),
       plot=nofrag_fitted_len_pos_plot,
       device="png",unit="cm",
       width=15,height=11)
nofrag_fitted_len_pos_plot
```



back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.16 - 0.88"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account

# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]

# take averages for positions
# don't want downward estimates influenced by return upward of U
# therefore, for downward influence, use only the values before the min
# take the difference between each value (differences between position proportion correct) **NOTE** pro
# average the difference in probabilities

# in case min is close to the end or we are not using a min (for non-U-shaped)
# functions, we need to get differences _first_ and then average those (e.g.
# if there is an effect of length and we just average position data,
# later positions) will have a lower average (because of the length effect)
# even if all position functions go upward
```

```
table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)
```

```
## [1] "mean change in probability for each additional length: "
```

```
print(OA_mean_len_diff)
```

```
## [1] -0.05068558
```

```
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding  U-shape): "
print(CurrentLabel)
```

```
## [1] "mean change in probability for each additional position (excluding  U-shape): "
```

```
print(OA_mean_pos_diff)
```

```
## [1] -0.02499948
```

```
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)
```

```
if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
                               2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
```

```
    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}
```

```
## [1] "Average upward change after U minimum"
## [1] 0.03572338
```

```
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upwa
```

```
  CurrentLabel<-"return upward value"
  print(upward_dist[biggest_return_upward_row])
   results_report_DF <- AddReportLine(results_report_DF,
                                      CurrentLabel,
                                      upward_dist[biggest_return_upward_row])

  print(" ")
  # percentage return upward
  percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upwar
  CurrentLabel <- "Return upward as a proportion of the downward distance:"
  print(CurrentLabel)
  print(percentage_return_upward)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                         percentage_return_upward)
}else{
  print("no U-shape in this participant")
}
```

```
## [1] "differences from left max to min for each row: "
## [1] 0.07512181 0.09757065 0.10319788 0.10439602 0.10329876 0.10000056 0.09477668
## [1] "differences from min to right max for each row: "
## [1] 0.00000000 0.00000000 0.00000000 0.01358052 0.04214872 0.08437654 0.13922749
## [1] " "
## [1] "row with biggest return upward"
## [1] 7
## [1] " "
## [1] "downward distance for row with the largest upward value"
## [1] 0.09477668
## [1] 0.1392275
## [1] " "
## [1] "Return upward as a proportion of the downward distance:"
## [1] 1.469006
```

```
FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small
            "preserved ~ stimlen*log_freq",
            "preserved ~ stimlen+log_freq",
            "preserved ~ pos*log_freq",
            "preserved ~ pos+log_freq",
            "preserved ~ stimlen*log_freq + pos*log_freq",
            "preserved ~ stimlen*log_freq + pos",
            "preserved ~ stimlen + pos*log_freq",
            "preserved ~ stimlen + pos + log_freq",
            "preserved ~ (I(pos^2)+pos)*log_freq",
            "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
            "preserved ~ stimlen*log_freq + I(pos^2) + pos",
            "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
            "preserved ~ stimlen + I(pos^2) + pos + log_freq",

            # models without frequency
            "preserved ~ 1",
            "preserved ~ stimlen",
            "preserved ~ pos",
            "preserved ~ stimlen + pos",
```

```
            "preserved ~ stimlen*pos",
            "preserved ~ I(pos^2)+pos",
            "preserved ~ stimlen + I(pos^2) + pos",
            "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)
```

```
## ****************************
## model index:  11
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq          I(pos^2)               pos  stimlen:log
##          1.74324          -0.18784           0.23657           0.03099          -0.28878          -0
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3566 Residual
## Null Deviance:       4922
## Residual Deviance: 4782  AIC: 4794
## log likelihood:  -2391.009
## Nagelkerke R2:  0.05140253
## % pres/err predicted correctly:  -1701.791
## % of predictable range [ (model-null)/(1-null) ]:  0.0385816
## ****************************
## model index:  10
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)            stimlen           log_freq           I(pos^2)                pos   stiml
##          1.759372          -0.189673           0.333845           0.032366          -0.296478
##      log_freq:pos
##         -0.040193
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3564 Residual
## Null Deviance:       4922
## Residual Deviance: 4779  AIC: 4795
## log likelihood:  -2389.659
## Nagelkerke R2:  0.05237353
## % pres/err predicted correctly:  -1700.566
## % of predictable range [ (model-null)/(1-null) ]:  0.03927295
## ****************************
## model index:  13
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos     log_freq
##     1.83353     -0.19573      0.03164     -0.29427      0.04880
##
```

```
## Degrees of Freedom: 3571 Total (i.e. Null);  3567 Residual
## Null Deviance:       4922
## Residual Deviance: 4787  AIC: 4797
## log likelihood:  -2393.503
## Nagelkerke R2:  0.04960534
## % pres/err predicted correctly:  -1703.966
## % of predictable range [ (model-null)/(1-null) ]:  0.0373534
## ***************************
## model index:  12
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##       (Intercept)            stimlen           I(pos^2)               pos           log_freq  I(pos
##          1.836914          -0.196206           0.031963          -0.295626           0.103193
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3565 Residual
## Null Deviance:       4922
## Residual Deviance: 4786  AIC: 4800
## log likelihood:  -2393.006
## Nagelkerke R2:  0.04996377
## % pres/err predicted correctly:  -1703.527
## % of predictable range [ (model-null)/(1-null) ]:  0.03760134
## ****************************
## model index:  20
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     1.95996     -0.21163      0.03191     -0.29622
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3568 Residual
## Null Deviance:       4922
## Residual Deviance: 4794  AIC: 4802
## log likelihood:  -2396.917
## Nagelkerke R2:  0.04714216
## % pres/err predicted correctly:  -1707.346
## % of predictable range [ (model-null)/(1-null) ]:  0.03544537
## ***************************
## model index:  21
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          I(pos^2)               pos  stimlen:I(pos^2)       stiml
##          1.09775          -0.10956          -0.05299           0.31926           0.00956          -0
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3566 Residual
## Null Deviance:       4922
## Residual Deviance: 4790  AIC: 4802
```

```
## log likelihood:  -2395.156
## Nagelkerke R2:  0.04841378
## % pres/err predicted correctly:  -1705.692
## % of predictable range [ (model-null)/(1-null) ]:  0.03637891
## **************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq               pos  stimlen:log_freq
##          1.19506          -0.16890           0.25126          -0.02760          -0.02599
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3567 Residual
## Null Deviance:       4922
## Residual Deviance: 4801  AIC: 4811
## log likelihood:  -2400.396
## Nagelkerke R2:  0.04462738
## % pres/err predicted correctly:  -1710.605
## % of predictable range [ (model-null)/(1-null) ]:  0.0336052
## **************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq  stimlen:log_freq
##          1.19278          -0.18224           0.25166          -0.02606
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3568 Residual
## Null Deviance:       4922
## Residual Deviance: 4803  AIC: 4811
## log likelihood:  -2401.684
## Nagelkerke R2:  0.04369457
## % pres/err predicted correctly:  -1711.882
## % of predictable range [ (model-null)/(1-null) ]:  0.03288379
## **************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq               pos  stimlen:log_freq      log_fr
##         1.195216         -0.169149          0.250874         -0.027144         -0.027082          0.
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3566 Residual
## Null Deviance:       4922
## Residual Deviance: 4801  AIC: 4813
## log likelihood:  -2400.363
## Nagelkerke R2:  0.04465079
## % pres/err predicted correctly:  -1710.571
```

```
## % of predictable range [ (model-null)/(1-null) ]:  0.03362424
## ***************************
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos     log_freq
##     1.27992     -0.17688     -0.02781      0.04994
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3568 Residual
## Null Deviance:       4922
## Residual Deviance: 4807  AIC: 4815
## log likelihood:  -2403.295
## Nagelkerke R2:  0.04252746
## % pres/err predicted correctly:  -1713.261
## % of predictable range [ (model-null)/(1-null) ]:  0.03210564
## ***************************
## model index:  18
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##     2.14330     -0.28331     -0.26951      0.02864
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3568 Residual
## Null Deviance:       4922
## Residual Deviance: 4807  AIC: 4815
## log likelihood:  -2403.439
## Nagelkerke R2:  0.04242356
## % pres/err predicted correctly:  -1713.4
## % of predictable range [ (model-null)/(1-null) ]:  0.03202704
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     log_freq
##     1.27788     -0.19035      0.04975
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3569 Residual
## Null Deviance:       4922
## Residual Deviance: 4809  AIC: 4815
## log likelihood:  -2404.602
## Nagelkerke R2:  0.04158014
## % pres/err predicted correctly:  -1714.568
## % of predictable range [ (model-null)/(1-null) ]:  0.03136772
## ***************************
## model index:  7
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##  (Intercept)       stimlen           pos      log_freq  pos:log_freq
##      1.27057      -0.17549      -0.02873       0.07118      -0.00556
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3567 Residual
## Null Deviance:       4922
## Residual Deviance: 4806  AIC: 4816
## log likelihood:  -2403.08
## Nagelkerke R2:  0.04268364
## % pres/err predicted correctly:  -1713.069
## % of predictable range [ (model-null)/(1-null) ]:  0.03221403
## ***************************
## model index:  17
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##     1.40438     -0.19295     -0.02753
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3569 Residual
## Null Deviance:       4922
## Residual Deviance: 4814  AIC: 4820
## log likelihood:  -2406.891
## Nagelkerke R2:  0.03991846
## % pres/err predicted correctly:  -1716.801
## % of predictable range [ (model-null)/(1-null) ]:  0.03010656
## ***************************
## model index:  15
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      1.4019      -0.2062
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3570 Residual
## Null Deviance:       4922
## Residual Deviance: 4816  AIC: 4820
## log likelihood:  -2408.174
## Nagelkerke R2:  0.03898668
## % pres/err predicted correctly:  -1718.102
## % of predictable range [ (model-null)/(1-null) ]:  0.02937228
## ***************************
## model index:  9
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
```

```
##
## Coefficients:
##       (Intercept)           I(pos^2)                pos           log_freq  I(pos^2):log_freq
##          0.374828           0.020229          -0.250259           0.159662           0.002483
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3566 Residual
## Null Deviance:       4922
## Residual Deviance: 4858  AIC: 4870
## log likelihood:  -2428.867
## Nagelkerke R2:  0.02385916
## % pres/err predicted correctly:  -1738.295
## % of predictable range [ (model-null)/(1-null) ]:  0.01797099
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos     log_freq
##     0.10446     -0.07677      0.08828
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3569 Residual
## Null Deviance:       4922
## Residual Deviance: 4868  AIC: 4874
## log likelihood:  -2434.151
## Nagelkerke R2:  0.01996842
## % pres/err predicted correctly:  -1743.533
## % of predictable range [ (model-null)/(1-null) ]:  0.01501358
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##  (Intercept)           pos      log_freq  pos:log_freq
##      0.10412      -0.07787       0.13243      -0.01175
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3568 Residual
## Null Deviance:       4922
## Residual Deviance: 4866  AIC: 4874
## log likelihood:  -2433.161
## Nagelkerke R2:  0.02069821
## % pres/err predicted correctly:  -1742.532
## % of predictable range [ (model-null)/(1-null) ]:  0.0155785
## ***************************
## model index:  19
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
```

```
##     0.40138      0.01951     -0.25186
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3569 Residual
## Null Deviance:       4922
## Residual Deviance: 4885  AIC: 4891
## log likelihood:  -2442.541
## Nagelkerke R2:  0.01376633
## % pres/err predicted correctly:  -1751.773
## % of predictable range [ (model-null)/(1-null) ]:  0.01036081
## ***************************
## model index:  16
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##     0.13748     -0.08444
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3570 Residual
## Null Deviance:       4922
## Residual Deviance: 4893  AIC: 4897
## log likelihood:  -2446.483
## Nagelkerke R2:  0.01084209
## % pres/err predicted correctly:  -1755.699
## % of predictable range [ (model-null)/(1-null) ]:  0.008144236
## ***************************
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##      -0.183
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3571 Residual
## Null Deviance:       4922
## Residual Deviance: 4922  AIC: 4924
## log likelihood:  -2461.025
## Nagelkerke R2:  2.968884e-16
## % pres/err predicted correctly:  -1770.124
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                          AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2
```

```
FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                       by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_freq_len_pos_model_summary.csv"),row.names = FALSE)
kable(FLPAICSummary)
```

Model	AIC	Delta AIC	Cum AIC	Num Par	(Intercept)	stimlen	log_freq	I(pos^2)	pos	log_freq:I(pos^2)	log_freq:pos	stimlen:log_freq	stimlen:I(pos^2)	stimlen:pos	stimlen:log_freq:I(pos^2)	stimlen:log_freq:pos
preserved ~ stimlen * log_freq + I(pos^2) + pos	4794.0	0.0	0.0	9	0.2365713	0.1878360	0.0242104	-0.2887797	NA	NA	0.0309879	NA	NA	NA	NA	NA
preserved ~ stimlen * log_freq + (I(pos^2) + pos) * log_freq	4795.3	1.3	1.3	11	0.3338447	0.1896730	0.0307709	-0.2964778	-0.0401934	NA	0.0323657	0.0055484	NA	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos + log_freq	4797.2	2.9	4.2	8	0.0487093	0.1957347	NA	-0.2942742	NA	NA	0.0316391	NA	NA	NA	NA	NA
preserved ~ stimlen + (I(pos^2) + pos) * log_freq	4800.5	3.2	7.5	10	0.1031033	0.1962064	NA	-0.2950258	-0.0325921	NA	0.0319636	0.0063364	NA	NA	NA	NA
preserved ~ stimlen + I(pos^2) + pos	4801.7	1.3	8.8	7	NA	0.2116294	NA	-0.2962176	NA	NA	0.0319765	NA	NA	NA	NA	NA
preserved ~ stimlen * (I(pos^2) + pos)	4802.8	1.1	9.9	9	NA	0.1095552	NA	0.3192592	NA	-0.0529853	NA	NA	-0.0709556	0.0095598	NA	NA
preserved ~ stimlen * log_freq + pos	4810.7	7.9	17.8	8	0.2512642	0.1688995	0.0259894	-0.2759556	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * log_freq	4811.3	0.6	18.4	7	0.2516552	0.1822443	0.0260620	NA	NA	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen * log_freq + pos * log_freq	4812.7	1.4	19.8	10	0.2508742	0.1691492	0.0270819	-0.2714433	NA	0.0023166	NA	NA	NA	NA	NA	NA

Model	AIC	DeltaAIC	AICwt	NagR2	(Intercept)	stimlen	log_freq	pos	stimlen:log_freq	log_freq:I(pos^2)	I(pos^2):pos	log_freq:pos	stimlen:I(pos^2)
preserved ~ stimlen + pos + log_freq	4814.2	20.5	7.1e-05	0.1768784	1.74324	-0.18784	0.23657	-0.2887800	NA	NA	NA	NA	NA
preserved ~ stimlen * pos	4814.3	20.7	6.4e-05	0.2833120	1.74324	-0.18784	0.23657	-0.2695120	NA	NA	NA	0.0286418	NA
preserved ~ stimlen + log_freq	4815.2	21.6	4.1e-05	0.1903520	1.74324	-0.18784	0.23657	-0.2887800	NA	NA	NA	NA	NA
preserved ~ stimlen + pos * log_freq	4816.2	22.5	2.5e-05	0.1754938	1.74324	-0.18784	0.23657	-0.2887800	-0.0287355600	NA	NA	NA	NA
preserved ~ stimlen + pos	4819.2	25.7	5.1e-06	0.1929503	1.74324	-0.18784	0.23657	-0.2887800	-0.0275328	NA	NA	NA	NA
preserved ~ stimlen	4820.2	26.7	3.1e-06	0.2062240	1.74324	-0.18784	0.23657	NA	NA	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos) * log_freq	4869.7	76.2	7.4e-17	0.1596617	1.74324	-0.18784	0.23657	-0.2887800	-0.0250859308450	0.0202932483232	NA	NA	NA
preserved ~ pos + log_freq	4874.3	80.8	7.4e-18	0.0882891	1.74324	-0.18784	0.23657	-0.2887800	-0.0767740	NA	NA	NA	NA
preserved ~ pos * log_freq	4874.3	80.8	7.4e-18	0.1324269	1.74324	-0.18784	0.23657	-0.2887800	-0.0778725217471	NA	NA	NA	NA
preserved ~ I(pos^2) + pos	4891.9	98.4	1.5e-21	0.1956777	1.74324	-0.18784	0.23657	-0.2887800	NA	0.0195777	NA	NA	NA
preserved ~ pos	4896.1	102.6	1.9e-22	0.1748435	1.74324	-0.18784	0.23657	-0.2887800	-0.0844419	NA	NA	NA	NA
preserved ~ 1	4924.1	130.6	1.5e-28	0.0000000	1.74324	-0.18784	0.23657	NA	NA	NA	NA	NA	NA

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ stimlen * log_freq + I(pos^2) + pos"
```

```
print(BestFLPModel)
```

```
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
##      (Intercept)           stimlen          log_freq          I(pos^2)               pos  stimlen:log
##          1.74324          -0.18784           0.23657           0.03099          -0.28878          -0
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3566 Residual
## Null Deviance:       4922
## Residual Deviance: 4782  AIC: 4794
```

```
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
```

```
PosDat$freq_bin[PosDat$log_freq >= median_freq]<-"hf"
PosDat$freq_bin[PosDat$log_freq <  median_freq]<-"lf"
```

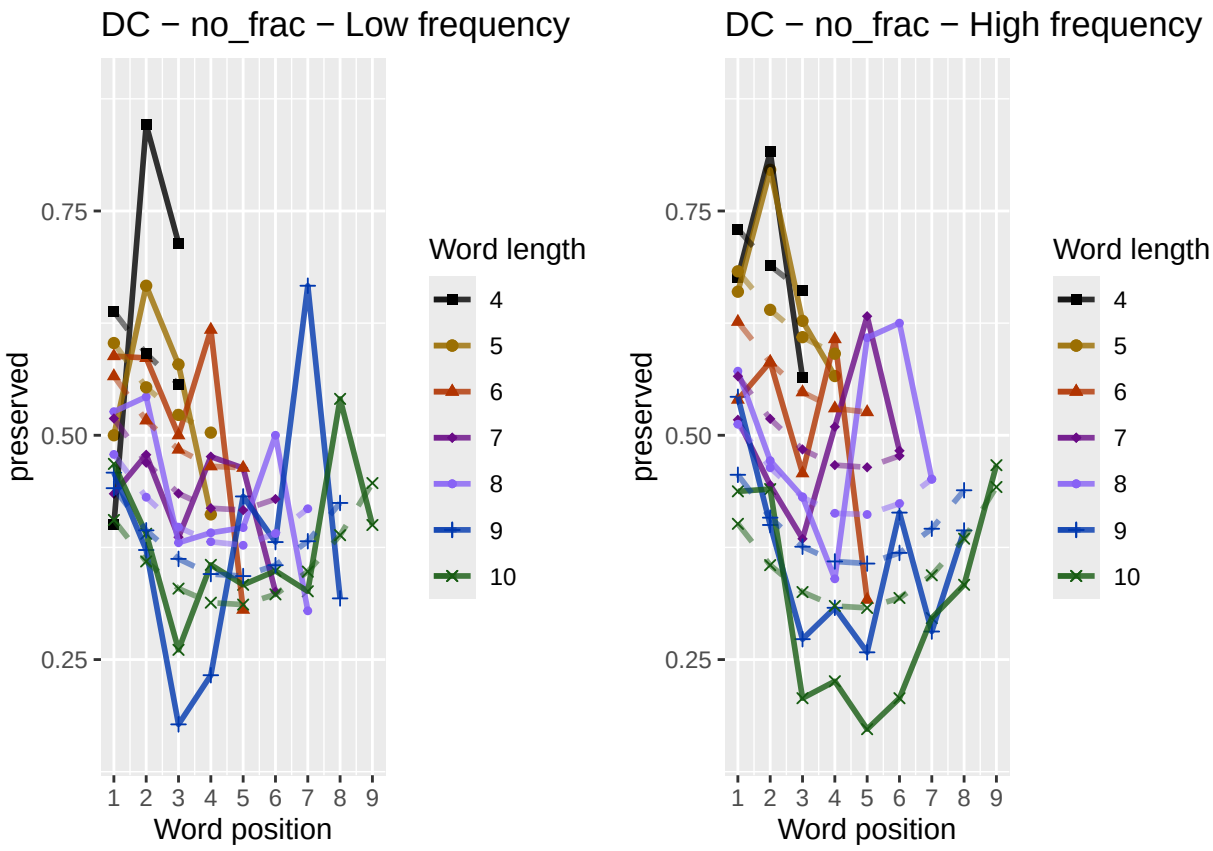
```
PosDat$FLPFitted<-fitted(BestFLPModel)

HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]

HF_Plot <- plot_len_pos_obs_predicted(HFDat,paste0(CurPat," - ",RunLabel," - High frequency"),"FLPFitte
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
LF_Plot <- plot_len_pos_obs_predicted(LFDat,paste0(CurPat," - ",RunLabel," - Low frequency"),"FLPFitted
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
library(ggpubr)
Both_Plots <- ggarrange(LF_Plot,HF_Plot) # labels=c("LF","HF",ncol=2)
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",
              CurTask,"_frequency_effect_length_pos_wfit.png"),
       device="png",unit="cm",width=30,height=11)
print(Both_Plots)
```



```r
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
MERes<-TestModels(MEModelEquations,PosDat)
```

```
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      1.4019      -0.2062
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3570 Residual
## Null Deviance:       4922
## Residual Deviance: 4816  AIC: 4820
## log likelihood:  -2408.174
## Nagelkerke R2:  0.03898668
## % pres/err predicted correctly:  -1718.102
## % of predictable range [ (model-null)/(1-null) ]:  0.02937228
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     0.40138      0.01951     -0.25186
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3569 Residual
## Null Deviance:       4922
## Residual Deviance: 4885  AIC: 4891
## log likelihood:  -2442.541
## Nagelkerke R2:  0.01376633
## % pres/err predicted correctly:  -1751.773
## % of predictable range [ (model-null)/(1-null) ]:  0.01036081
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##     0.13748     -0.08444
```

```
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3570 Residual
## Null Deviance:       4922
## Residual Deviance: 4893  AIC: 4897
## log likelihood:  -2446.483
## Nagelkerke R2:  0.01084209
## % pres/err predicted correctly:  -1755.699
## % of predictable range [ (model-null)/(1-null) ]:  0.008144236
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##    -0.01993     -0.11802
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3570 Residual
## Null Deviance:       4922
## Residual Deviance: 4896  AIC: 4900
## log likelihood:  -2448.116
## Nagelkerke R2:  0.009629446
## % pres/err predicted correctly:  -1757.281
## % of predictable range [ (model-null)/(1-null) ]:  0.007251205
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##      -0.183
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3571 Residual
## Null Deviance:       4922
## Residual Deviance: 4922  AIC: 4924
## log likelihood:  -2461.025
## Nagelkerke R2:  2.968884e-16
## % pres/err predicted correctly:  -1770.124
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##    -0.22142      0.03286
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3570 Residual
## Null Deviance:       4922
```

```
## Residual Deviance: 4921  AIC: 4925
## log likelihood:  -2460.334
## Nagelkerke R2:  0.0005169456
## % pres/err predicted correctly:  -1769.438
## % of predictable range [ (model-null)/(1-null) ]:  0.0003870306
## ***************************
```

```
BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

MEAICSummary<-data.frame(Model=MERes$Model,
                         AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                          by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                              CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ stimlen	4820.347	0.00000	1	1	0.0389867	1.4018558	NA	NA	NA	NA	-0.206224
preserved ~ (I(pos^2) + pos)	4891.082	70.73469	0	0	0.0137660	0.4013786	NA	NA	0.0195077	-0.2518602	NA
preserved ~ pos	4896.967	76.61941	0	0	0.0108420	0.1374835	NA	NA	NA	-0.0844419	NA
preserved ~ CumErr	4900.231	79.88401	0	0	0.0096294	-0.0199340	NA	-0.1180226	NA	NA	NA
preserved ~ 1	4924.049	103.70235	0	0	0.0000000	-0.1830401	NA	NA	NA	NA	NA
preserved ~ CumPres	4924.668	104.32106	0	0	0.0005169	-0.2214181	0.0328649	NA	NA	NA	NA

```
if(DoSimulations){
      BestMEModelFormulaRnd <- BestMEModelFormula
    if(grepl("CumPres",BestMEModelFormulaRnd)){
      BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
    }else if(grepl("CumErr",BestMEModelFormulaRnd)){
      BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
    }

    RndModelAIC<-numeric(length=RandomSamples)
    for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
      PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
      PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
```

```
      BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                          family="binomial",data=PosDat)
      RndModelAIC[rindex] <- BestModelRnd$aic
    }
    ModelNames<-c(paste0("***",BestMEModelFormula),
                  rep(BestMEModelFormulaRnd,RandomSamples))
    AICValues <- c(BestMEModel$aic,RndModelAIC)
    BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
    BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
    BestMEModelRndDF <- rbind(BestMEModelRndDF,
                              data.frame(Name=c("Random average"),
                                         AIC=c(mean(RndModelAIC))))
    BestMEModelRndDF <- rbind(BestMEModelRndDF,
                              data.frame(Name=c("Random SD"),
                                         AIC=c(sd(RndModelAIC))))

    write.csv(BestMEModelRndDF,
              paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                     "_best_main_effects_model_with_random_cum_term.csv"),
              row.names = FALSE)
}
```

```
syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary,paste0(TablesDir,CurPat,"_",
                                        RunLabel,"_",CurTask,
                                        "_syllable_component_summary.csv"),row.names = FALSE)
kable(syll_component_summary)
```

syll_component	MeanPres	N
1	0.3375796	471
O	0.4009324	1716
P	0.1428571	28
S	0.3737864	206
V	0.6038228	1151

```
#  main effects models for data without satellite positions

keep_components = c("O","V","1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                              stimlen,stim,pos,
                              preserved,syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data,MEModelEquations)
```

```
## ***************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      1.4415      -0.2072
##
## Degrees of Freedom: 3337 Total (i.e. Null);  3336 Residual
## Null Deviance:       4608
## Residual Deviance: 4508  AIC: 4512
## log likelihood:  -2253.976
## Nagelkerke R2:  0.0394869
## % pres/err predicted correctly:  -1608.968
## % of predictable range [ (model-null)/(1-null) ]:  0.02975198
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     0.55777      0.02521     -0.31302
##
## Degrees of Freedom: 3337 Total (i.e. Null);  3335 Residual
## Null Deviance:       4608
## Residual Deviance: 4560  AIC: 4566
## log likelihood:  -2280.003
## Nagelkerke R2:  0.01911198
## % pres/err predicted correctly:  -1634.423
## % of predictable range [ (model-null)/(1-null) ]:  0.01441102
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##     0.03679     -0.15119
##
## Degrees of Freedom: 3337 Total (i.e. Null);  3336 Residual
## Null Deviance:       4608
## Residual Deviance: 4572  AIC: 4576
## log likelihood:  -2286.065
## Nagelkerke R2:  0.01432012
## % pres/err predicted correctly:  -1640.448
## % of predictable range [ (model-null)/(1-null) ]:  0.01078053
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
```

```
## (Intercept)          pos
##     0.21564     -0.09643
##
## Degrees of Freedom: 3337 Total (i.e. Null);  3336 Residual
## Null Deviance:       4608
## Residual Deviance: 4573  AIC: 4577
## log likelihood:  -2286.254
## Nagelkerke R2:  0.01417069
## % pres/err predicted correctly:  -1640.627
## % of predictable range [ (model-null)/(1-null) ]:  0.0106725
## ***************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##     -0.1525
##
## Degrees of Freedom: 3337 Total (i.e. Null);  3337 Residual
## Null Deviance:       4608
## Residual Deviance: 4608  AIC: 4610
## log likelihood:  -2304.052
## Nagelkerke R2:  -2.966342e-16
## % pres/err predicted correctly:  -1658.336
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##    -0.16586      0.01191
##
## Degrees of Freedom: 3337 Total (i.e. Null);  3336 Residual
## Null Deviance:       4608
## Residual Deviance: 4608  AIC: 4612
## log likelihood:  -2303.971
## Nagelkerke R2:  6.483045e-05
## % pres/err predicted correctly:  -1658.256
## % of predictable range [ (model-null)/(1-null) ]:  4.857529e-05
## ***************************
```

```
write.csv(SimpSyllMEAICSummary,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",
                 CurTask,
                 "_OV1_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ stimlen	4511.953	0.00000	1	1	0.0394869	1.4415354	NA	NA	NA	NA	-0.2072382
preserved ~ (I(pos^2) + pos)	4566.005	54.05242	0	0	0.0191120	0.5577653	NA	NA	0.0252109	-0.3130159	NA
preserved ~ CumErr	4576.136	64.17734	0	0	0.0143200	1.0367910	NA	-0.1511929	NA	NA	NA
preserved ~ pos	4576.507	64.55473	0	0	0.0141707	0.2156445	NA	NA	NA	-0.0964315	NA
preserved ~ 1	4610.104	98.15144	0	0	0.0000000	-0.1524817	NA	NA	NA	NA	NA
preserved ~ CumPres	4611.942	99.98945	0	0	0.0000648	-0.1658619	0.0119102	NA	NA	NA	NA

```
#  main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
#  also reduces data)

keep_components = c("O","V")
OVData <- PosDat[PosDat$syll_component %in% keep_components,]
OVData <- OVData %>% select(stim_number,
                            stimlen,stim,pos,
                            preserved,syll_component)
OVData$CumPres <- CalcCumPres(OVData)
OVData$CumErr <- CalcCumErrFromPreserved(OVData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVData,MEModelEquations)
```

```
## ****************************
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      1.4509      -0.1982
##
## Degrees of Freedom: 2866 Total (i.e. Null);  2865 Residual
## Null Deviance:       3971
## Residual Deviance: 3891  AIC: 3895
## log likelihood:  -1945.638
## Nagelkerke R2:  0.03655746
## % pres/err predicted correctly:  -1391.334
## % of predictable range [ (model-null)/(1-null) ]:  0.02750999
## ****************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
```

```
##     0.56689      0.01935     -0.26644
##
## Degrees of Freedom: 2866 Total (i.e. Null);  2864 Residual
## Null Deviance:       3971
## Residual Deviance: 3928  AIC: 3934
## log likelihood:  -1964.159
## Nagelkerke R2:  0.01968696
## % pres/err predicted correctly:  -1409.472
## % of predictable range [ (model-null)/(1-null) ]:  0.01484133
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      0.3135      -0.1011
##
## Degrees of Freedom: 2866 Total (i.e. Null);  2865 Residual
## Null Deviance:       3971
## Residual Deviance: 3935  AIC: 3939
## log likelihood:  -1967.541
## Nagelkerke R2:  0.01658211
## % pres/err predicted correctly:  -1412.833
## % of predictable range [ (model-null)/(1-null) ]:  0.0124937
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)       CumErr
##     0.08683     -0.15627
##
## Degrees of Freedom: 2866 Total (i.e. Null);  2865 Residual
## Null Deviance:       3971
## Residual Deviance: 3947  AIC: 3951
## log likelihood:  -1973.431
## Nagelkerke R2:  0.0111587
## % pres/err predicted correctly:  -1418.698
## % of predictable range [ (model-null)/(1-null) ]:  0.008397908
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##    -0.01431     -0.05745
##
## Degrees of Freedom: 2866 Total (i.e. Null);  2865 Residual
```

```
## Null Deviance:       3971
## Residual Deviance: 3968  AIC: 3972
## log likelihood:  -1984.138
## Nagelkerke R2:  0.00124215
## % pres/err predicted correctly:  -1429.391
## % of predictable range [ (model-null)/(1-null) ]:  0.0009290014
## ****************************
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##    -0.07049
##
## Degrees of Freedom: 2866 Total (i.e. Null);  2866 Residual
## Null Deviance:       3971
## Residual Deviance: 3971  AIC: 3973
## log likelihood:  -1985.474
## Nagelkerke R2:  -2.961821e-16
## % pres/err predicted correctly:  -1430.721
## % of predictable range [ (model-null)/(1-null) ]:  0
## ****************************
```

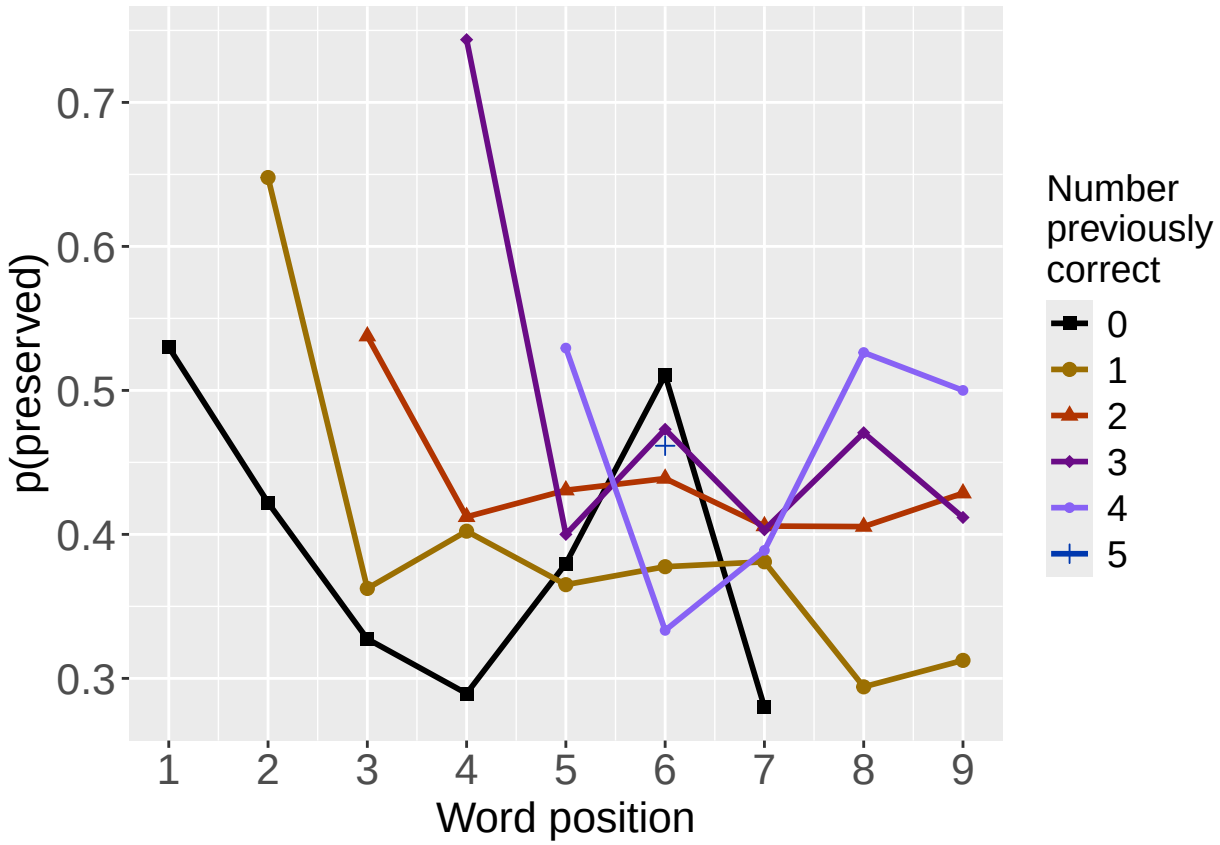
```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                 "_OV_main_effects_model_summary.csv"),row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ stimlen	3895.275	0.00000	1	1	0.0365575	1.4508752	NA	NA	NA	NA	-0.1981673
preserved ~ (I(pos^2) + pos)	3934.317	39.04225	0	0	0.0196870	0.5668913	NA	NA	0.0193526	-0.2664361	NA
preserved ~ pos	3939.083	43.80767	0	0	0.0165820	0.3134557	NA	NA	NA	-0.1011034	NA
preserved ~ CumErr	3950.862	55.58702	0	0	0.0111587	0.0868334	NA	-0.1562694	NA	NA	NA
preserved ~ CumPres	3972.276	77.00090	0	0	0.0012421	-0.0143059	0.0574479	NA	NA	NA	NA
preserved ~ 1	3972.947	77.67197	0	0	0.0000000	-0.0704861	NA	NA	NA	NA	NA

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

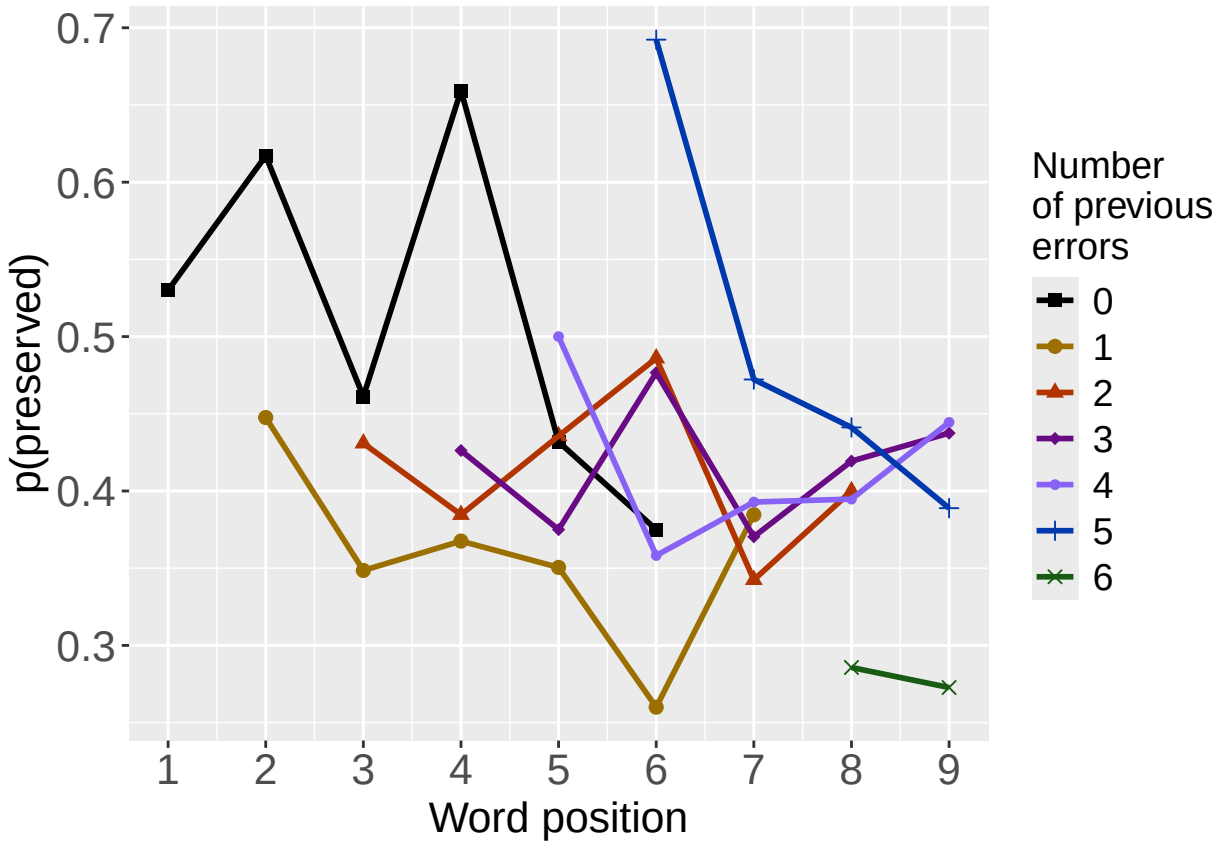
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

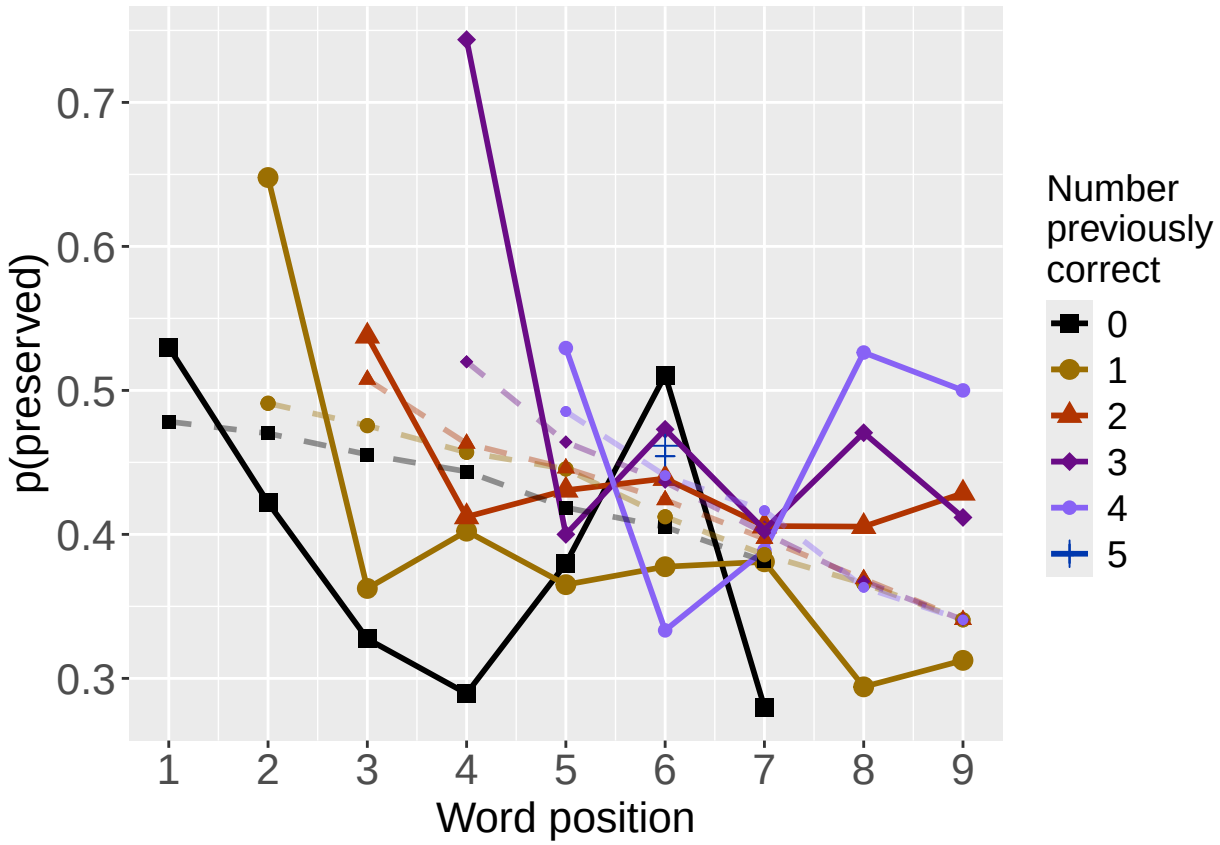
```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

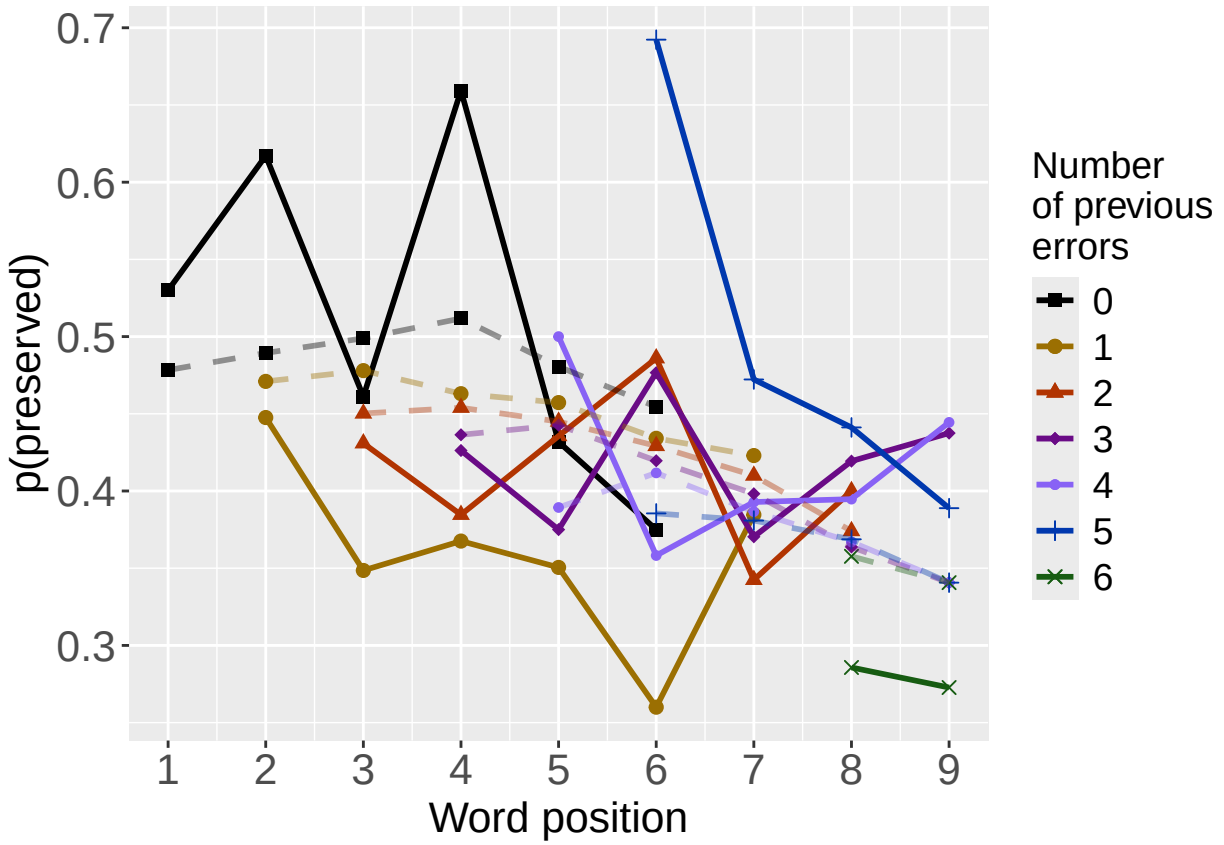
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                 CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                 CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")
```

```
## Saving 6.5 x 4.5 in image
```

```
CumAICSummary <- NULL
```

```
########
# level 2 -- Add position squared (quadratic with position)
########

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     1.95996     -0.21163      0.03191     -0.29622
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3568 Residual
## Null Deviance:       4922
## Residual Deviance: 4794  AIC: 4802
## log likelihood:  -2396.917
## Nagelkerke R2:  0.04714216
## % pres/err predicted correctly:  -1707.346
## % of predictable range [ (model-null)/(1-null) ]:  0.03544537
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      1.4019      -0.2062
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3570 Residual
## Null Deviance:       4922
## Residual Deviance: 4816  AIC: 4820
## log likelihood:  -2408.174
## Nagelkerke R2:  0.03898668
## % pres/err predicted correctly:  -1718.102
## % of predictable range [ (model-null)/(1-null) ]:  0.02937228
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)     I(pos^2)          pos
##     0.40138      0.01951     -0.25186
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3569 Residual
## Null Deviance:       4922
## Residual Deviance: 4885  AIC: 4891
## log likelihood:  -2442.541
## Nagelkerke R2:  0.01376633
## % pres/err predicted correctly:  -1751.773
## % of predictable range [ (model-null)/(1-null) ]:  0.01036081
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	I(pos^2)	pos
preserved ~ stimlen + I(pos^2) + pos	4801.834	0.00000	1.00e+00	0.9999045	0.0471422	1.9599583	-0.2116294	0.0319065	-0.2962176
preserved ~ stimlen	4820.347	18.51272	9.55e-05	0.0000955	0.0389867	1.4018558	-0.2062240	NA	NA
preserved ~ I(pos^2) + pos	4891.082	89.24741	0.00e+00	0.0000000	0.0137663	0.4013786	NA	0.0195077	-0.2518602

```
########
# level 2 -- Add length
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
########
# level 2 -- add cumulative preserved
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      CumPres
##     1.36428     -0.21122      0.06487
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3569 Residual
## Null Deviance:       4922
## Residual Deviance: 4811  AIC: 4817
## log likelihood:  -2405.586
## Nagelkerke R2:  0.04086572
## % pres/err predicted correctly:  -1715.637
## % of predictable range [ (model-null)/(1-null) ]:  0.03076389
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      1.4019      -0.2062
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3570 Residual
```

```
## Null Deviance:       4922
## Residual Deviance: 4816  AIC: 4820
## log likelihood:  -2408.174
## Nagelkerke R2:  0.03898668
## % pres/err predicted correctly:  -1718.102
## % of predictable range [ (model-null)/(1-null) ]:  0.02937228
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##    -0.22142      0.03286
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3570 Residual
## Null Deviance:       4922
## Residual Deviance: 4921  AIC: 4925
## log likelihood:  -2460.334
## Nagelkerke R2:  0.0005169456
## % pres/err predicted correctly:  -1769.438
## % of predictable range [ (model-null)/(1-null) ]:  0.0003870306
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	CumPres
preserved ~ stimlen + CumPres	4817.173	0.000000	1.0000000	0.8302224	0.0408657	1.3642807	-0.2112209	0.0648654
preserved ~ stimlen	4820.347	3.174409	0.2044965	0.1697776	0.0389867	1.4018558	-0.2062240	NA
preserved ~ CumPres	4924.668	107.495465	0.0000000	0.0000000	0.0005169	-0.2214181	NA	0.0328649

```
########
# level 2 -- Add linear position (NOT quadratic)
########

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen          pos
##     1.40438     -0.19295     -0.02753
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3569 Residual
## Null Deviance:       4922
## Residual Deviance: 4814  AIC: 4820
## log likelihood:  -2406.891
## Nagelkerke R2:  0.03991846
## % pres/err predicted correctly:  -1716.801
## % of predictable range [ (model-null)/(1-null) ]:  0.03010656
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      1.4019      -0.2062
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3570 Residual
## Null Deviance:       4922
## Residual Deviance: 4816  AIC: 4820
## log likelihood:  -2408.174
## Nagelkerke R2:  0.03898668
## % pres/err predicted correctly:  -1718.102
## % of predictable range [ (model-null)/(1-null) ]:  0.02937228
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##     0.13748     -0.08444
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3570 Residual
## Null Deviance:       4922
## Residual Deviance: 4893  AIC: 4897
## log likelihood:  -2446.483
## Nagelkerke R2:  0.01084209
## % pres/err predicted correctly:  -1755.699
## % of predictable range [ (model-null)/(1-null) ]:  0.008144236
## ***************************
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos
preserved ~ stimlen + pos	4819.782	0.0000000	1.0000000	0.5701545	0.0399185	1.4043845	-0.1929503	-0.0275328
preserved ~ stimlen	4820.347	0.5649633	0.7539105	0.4298455	0.0389867	1.4018558	-0.2062240	NA
preserved ~ pos	4896.967	77.1843754	0.0000000	0.0000000	0.0108421	0.1374835	NA	-0.0844419

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_main_effects_plus_one_model_summary.csv"),row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	I(pos^2)	pos	CumPres
preserved ~ stimlen + I(pos^2) + pos	4801.834	0.0000000	1.0000000	0.9999046	0.0471422	2.9599583	-0.2116294	0.0319065	-0.2962176	NA
preserved ~ stimlen + CumPres	4817.173	0.0000000	1.0000000	0.8302220	0.0408657	7.3642807	-0.2112209	NA	NA	0.0648654
preserved ~ stimlen + pos	4819.782	0.0000000	1.0000000	0.5701545	0.0399185	5.4043845	-0.1929503	NA	-0.0275328	NA
preserved ~ stimlen	4820.347	18.5127176	0.0000956	0.0000956	0.0389867	7.4018558	-0.2062240	NA	NA	NA
preserved ~ stimlen	4820.347	3.1744089	0.2044965	0.1697776	0.0389867	7.4018558	-0.2062240	NA	NA	NA
preserved ~ stimlen	4820.347	0.5649633	0.7539105	0.4298456	0.0389867	7.4018558	-0.2062240	NA	NA	NA
preserved ~ I(pos^2) + pos	4891.082	89.2474050	0.0000000	0.0000000	0.0137668	3.4013786	NA	0.0195077	-0.2518602	NA
preserved ~ pos	4896.967	77.1843754	0.0000000	0.0000000	0.0108420	0.1374835	NA	NA	-0.0844419	NA
preserved ~ CumPres	4924.668	107.4954650	0.0000000	0.0000000	0.0005169	-0.2214181	NA	NA	NA	0.0328649

```
# explore influence of frequency and length

if(grepl("stimlen",BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + log_freq")
  )
}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
    Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
      BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
      paste0(BestModelFormulaL2," + log_freq"),
      paste0(BestModelFormulaL2," + stimlen"),
      paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)
```

```
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos     log_freq
##     1.83353     -0.19573      0.03164     -0.29427      0.04880
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3567 Residual
## Null Deviance:       4922
## Residual Deviance: 4787  AIC: 4797
## log likelihood:  -2393.503
## Nagelkerke R2:  0.04960534
## % pres/err predicted correctly:  -1703.966
## % of predictable range [ (model-null)/(1-null) ]:  0.0373534
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen     I(pos^2)          pos
##     1.95996     -0.21163      0.03191     -0.29622
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3568 Residual
## Null Deviance:       4922
## Residual Deviance: 4794  AIC: 4802
## log likelihood:  -2396.917
## Nagelkerke R2:  0.04714216
## % pres/err predicted correctly:  -1707.346
## % of predictable range [ (model-null)/(1-null) ]:  0.03544537
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)
##      -0.183
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3571 Residual
## Null Deviance:       4922
## Residual Deviance: 4922  AIC: 4924
## log likelihood:  -2461.025
## Nagelkerke R2:  2.968884e-16
## % pres/err predicted correctly:  -1770.124
## % of predictable range [ (model-null)/(1-null) ]:  0
## ***************************
```

```
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

AICSummary<-data.frame(Model=Level3Res$Model,
                       AIC=Level3Res$AIC,
                       row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                          by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                            CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	I(pos^2)	pos	log_freq
preserved ~ stimlen + I(pos^2) + pos + log_freq	4797.007	0.000000	1.000000	0.9178673	0.0496053	3.8335342	-0.1957347	0.0316391	-0.2942742	0.0487993
preserved ~ stimlen + I(pos^2) + pos	4801.834	4.827433	0.0894821	0.0821327	0.0471422	2.9599583	-0.2116294	0.0319065	-0.2962176	NA
preserved ~ 1	4924.049	127.042504	0.0000000	0.0000000	0.0000000	-0.1830401	NA	NA	NA	NA

```
# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions
```

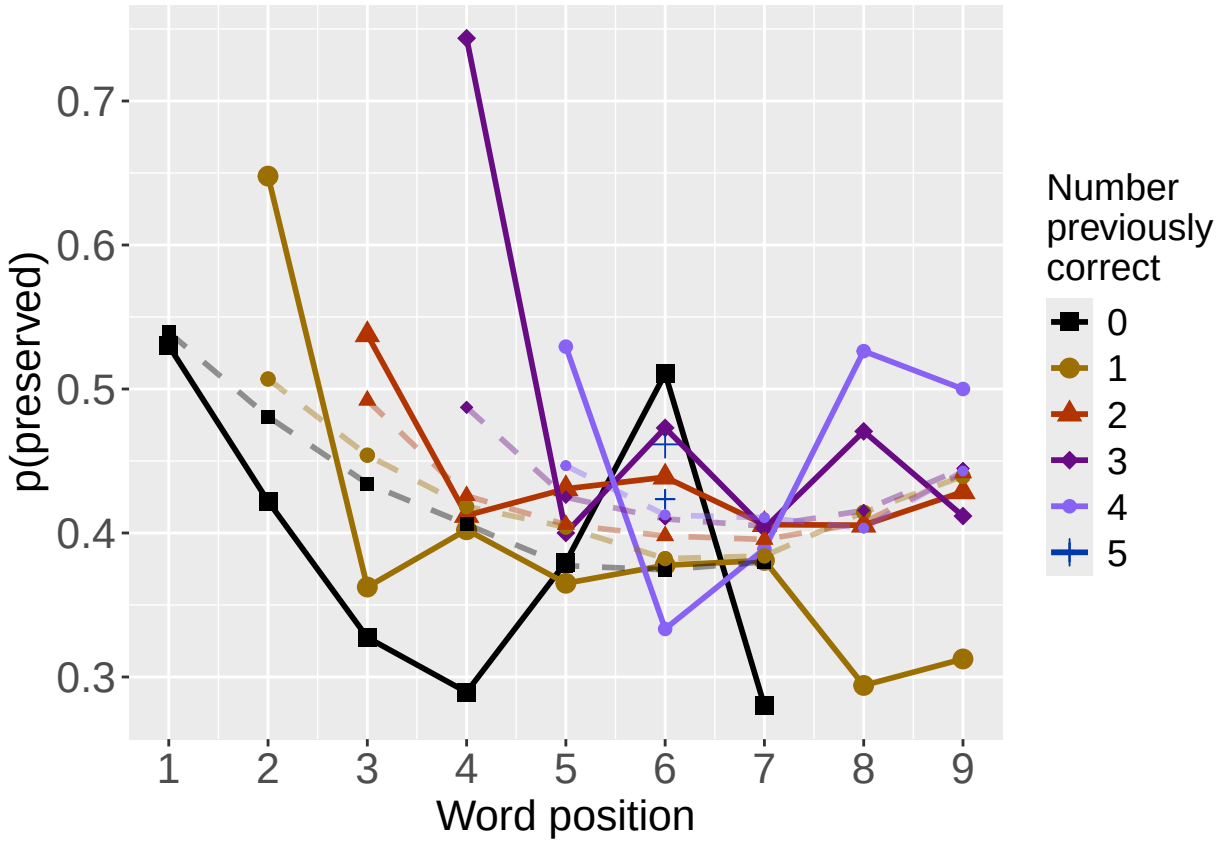
```
## Single term deletions
##
## Model:
## preserved ~ stimlen + I(pos^2) + pos + log_freq
##          Df Deviance    AIC
## stimlen   1   4859.5 4867.5
## pos       1   4809.1 4817.1
## I(pos^2)  1   4806.6 4814.6
## log_freq  1   4793.8 4801.8
## <none>        4787.0 4797.0
```

```
################################
# Single deletions from best model
################################

write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                    RunLabel,"_",CurTask,
                                    "_best_model_single_term_deletions.csv"),
          row.names = TRUE)
```

```
# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

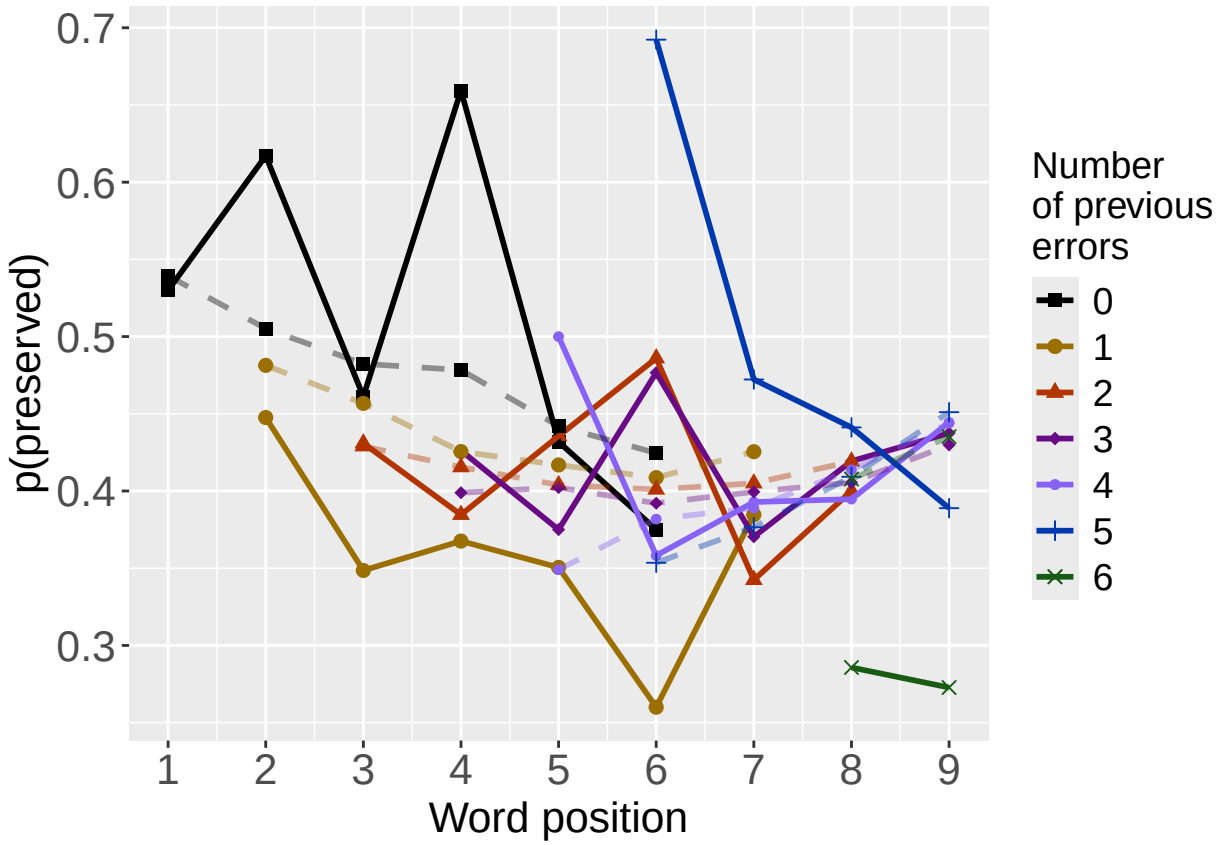
```
print(PrevCorPlot)
```




```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```




```
PlotName<-paste0(FigDir,CurPat,"_",
                 RunLabel,"_",CurTask,
                 "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =
```

```
## Saving 6.5 x 4.5 in image
```

```
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "
```

```
## Saving 6.5 x 4.5 in image
```

```
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
  # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                        family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
```

```
  }
  ModelNames<-c(paste0("***",BestModelFormulaL3),
                rep(BestModelFormulaL3Rnd,RandomSamples))
  AICValues <- c(BestModelL3$aic,RndModelAIC)
  BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
  BestModelRndDF <- rbind(BestModelRndDF,
                          data.frame(Name=c("Random average"),
                                     AIC=c(mean(RndModelAIC))))
  BestModelRndDF <- rbind(BestModelRndDF,
                          data.frame(Name=c("Random SD"),
                                     AIC=c(sd(RndModelAIC))))
  write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                  "_best_model_with_random_cum_term.csv"),
            row.names = FALSE)
}
```

```
# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)
```

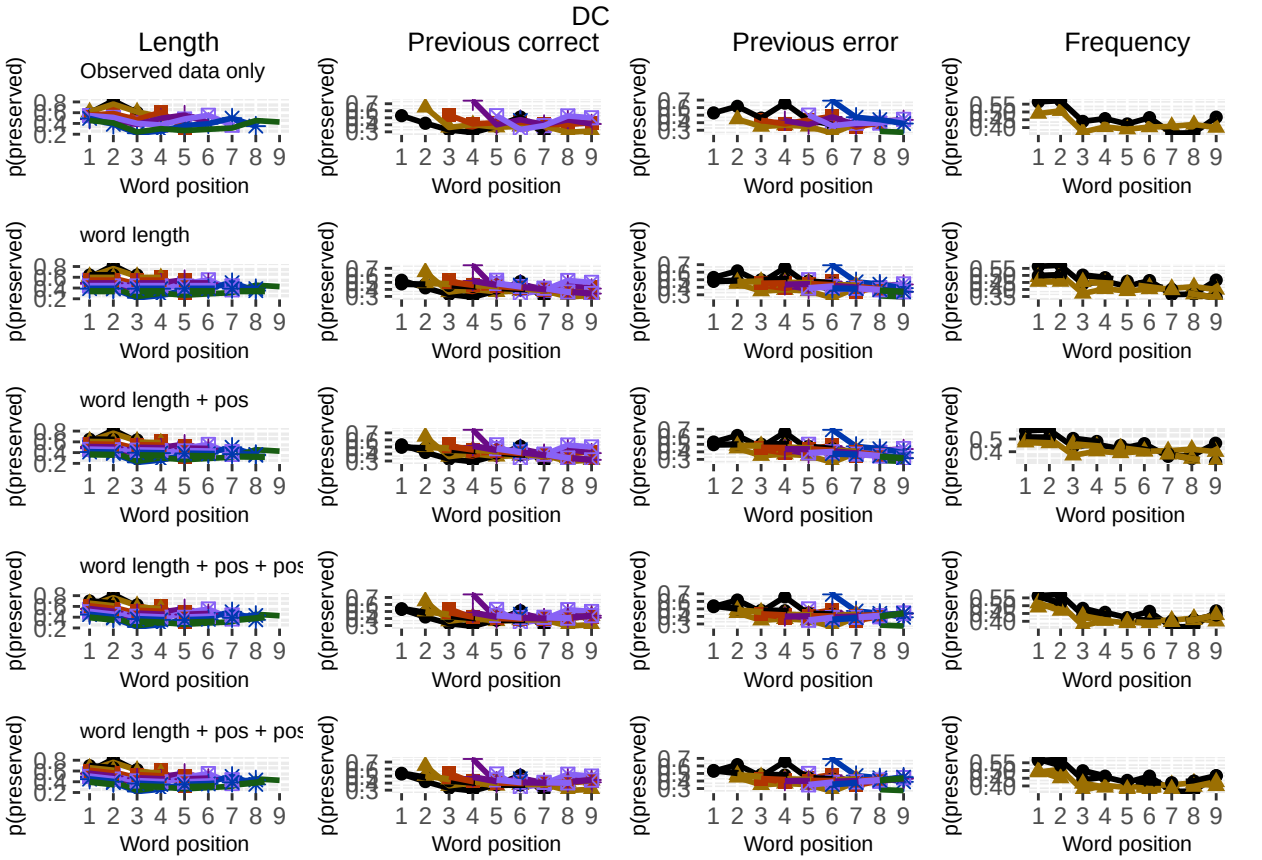
```
## ***************************
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      1.4019      -0.2062
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3570 Residual
## Null Deviance:       4922
## Residual Deviance: 4816  AIC: 4820
## log likelihood:  -2408.174
## Nagelkerke R2:  0.03898668
## % pres/err predicted correctly:  -1718.102
## % of predictable range [ (model-null)/(1-null) ]:  0.02937228
```

```
## ***************************
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##     1.40438     -0.19295     -0.02753
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3569 Residual
## Null Deviance:       4922
## Residual Deviance: 4814  AIC: 4820
## log likelihood:  -2406.891
## Nagelkerke R2:  0.03991846
## % pres/err predicted correctly:  -1716.801
## % of predictable range [ (model-null)/(1-null) ]:  0.03010656
## ***************************
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos     I(pos^2)
##     1.95996     -0.21163     -0.29622      0.03191
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3568 Residual
## Null Deviance:       4922
## Residual Deviance: 4794  AIC: 4802
## log likelihood:  -2396.917
## Nagelkerke R2:  0.04714216
## % pres/err predicted correctly:  -1707.346
## % of predictable range [ (model-null)/(1-null) ]:  0.03544537
## ***************************
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##     data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos     I(pos^2)     log_freq
##     1.83353     -0.19573     -0.29427      0.03164      0.04880
##
## Degrees of Freedom: 3571 Total (i.e. Null);  3567 Residual
## Null Deviance:       4922
## Residual Deviance: 4787  AIC: 4797
## log likelihood:  -2393.503
## Nagelkerke R2:  0.04960534
## % pres/err predicted correctly:  -1703.966
## % of predictable range [ (model-null)/(1-null) ]:  0.0373534
## ***************************
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'freq_bin'. You can override using the `.groups` argument.

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 2 rows containing missing values or values outside the scale range (`geom_point()`)

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 2 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 2 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 2 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 2 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
```

```
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 2 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 2 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`).

## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.

## Warning: Removed 2 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 2 rows containing missing values or values outside the scale range (`geom_point()`).
```

```
ggsave(paste0(PlotName,".tif"),plot=FactorPlot,width = 360,height=400,units="mm",device="tiff",compressi
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```




```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                       CurTask,"_dominance_analysis_table.csv"),
          row.names = TRUE)
kable(DAContributionAverage)
```

	stimlen	I(pos^2)	pos	log_freq
McFadden	0.0169676	0.0029129	0.0040339	0.0035219

	stimlen	I(pos^2)	pos	log_freq
SquaredCorrelation	0.0229090	0.0039384	0.0054615	0.0047912
Nagelkerke	0.0229090	0.0039384	0.0054615	0.0047912
Estrella	0.0232490	0.0039926	0.0055313	0.0048358

	deviance	deviance_explained
stimlen + pos + I(pos^2) + log_freq	4787.007	135.0425
stimlen + pos + I(pos^2)	4793.834	128.2151
stimlen + pos	4813.782	108.2673
stimlen	4816.347	105.7024
null	4922.049	0.0000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                                                 model deviance deviance_explained percent_explained
## stimlen + pos + I(pos^2) + log_freq stimlen + pos + I(pos^2) + log_freq 4787.007           135.0425          2.743623
## stimlen + pos + I(pos^2)                       stimlen + pos + I(pos^2) 4793.834           128.2151          2.604912
## stimlen + pos                                        stimlen + pos 4813.782           108.2673          2.199639
## stimlen                                                    stimlen 4816.347           105.7024          2.147527
## null                                                          null 4922.049             0.0000          0.000000
##                                     percent_of_explained_deviance increment_in_explained
## stimlen + pos + I(pos^2) + log_freq                     100.00000              5.055766
## stimlen + pos + I(pos^2)                                 94.94423             14.771464
## stimlen + pos                                            80.17277              1.899375
## stimlen                                                  78.27340             78.273396
## null                                                           NA              0.000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
```

	percent_explained	percent_of_explained_deviance	increment_in_explained
stimlen + pos + I(pos^2) + log_freq	2.743623	100.00000	5.055766
stimlen + pos + I(pos^2)	2.604912	94.94423	14.771463
stimlen + pos	2.199639	80.17277	1.899375
stimlen	2.147527	78.27340	78.273396
null	0.000000	NA	0.000000

```
                                  "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents
```

```
##           Nagelkerke
## stimlen   0.6174912
## I(pos^2)  0.1061554
## pos       0.1472098
## log_freq  0.1291436
```

```
sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```



```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
```

```
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumerr_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length
```

```
sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_sse_results_table.csv"),row.names = TRUE)
sse_table
```

```
##                                    model p_accounted_for model_deviance diff_stimlen diff_stimlen+pos+I(pos^2) diff_stimlen+pos
## 1                      preserved ~ stimlen       0.3020085       4816.347   0.00000000               -0.01743149     -0.030799940
## 2          preserved ~ stimlen+pos+I(pos^2)       0.3194400       4793.834   0.01743149                0.00000000     -0.013368446
## 3                   preserved ~ stimlen+pos       0.3328084       4813.782   0.03079994                0.01336845      0.000000000
```

model	p_accounted_for	model_deviance
preserved ~ stimlen	0.3020085	4816.347
preserved ~ stimlen+pos+I(pos^2)	0.3194400	4793.834
preserved ~ stimlen+pos	0.3328084	4813.782
preserved ~ stimlen+pos+I(pos^2)+log_freq	0.3346412	4787.007

model	diff_stimlen	diff_stimlen+pos+I(pos^2)	diff_stimlen+pos	diff_stimlen+pos+I(pos^2)+log_freq
preserved ~ stimlen	0.0000000	-0.0174315	-0.0307999	-0.0326328
preserved ~ stimlen+pos+I(pos^2)	0.0174315	0.0000000	-0.0133684	-0.0152013
preserved ~ stimlen+pos	0.0307999	0.0133684	0.0000000	-0.0018328
preserved ~ stimlen+pos+I(pos^2)+log_freq	0.0326328	0.0152013	0.0018328	0.0000000

```
## 4 preserved ~ stimlen+pos+I(pos^2)+log_freq       0.3346412       4787.007   0.03263275                0.01520126       0.001832813
##   diff_stimlen+pos+I(pos^2)+log_freq
## 1                       -0.032632753
## 2                       -0.015201259
## 3                       -0.001832813
## 4                        0.000000000
```

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                   CurTask,"_results_report_df.csv"),row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```